



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

Proyecto Syntax Validator

Curso: Base de Datos II

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Arocutipa Arocutipa, Gian Franco* *(2023076790)

Soto Oquendo Cristian Gabriel* *(2026086510)

**Tacna – Perú
2026**

Syntax Validator
Informe de Proyecto
Versión {1.0}

Informe Proyecto Final

1. Antecedentes

El manejo de bases de datos constituye una competencia fundamental en la formación de ingenieros de sistemas y profesionales del software. En el ecosistema académico y profesional actual, el lenguaje SQL y las interfaces de consulta NoSQL —particularmente MongoDB— representan dos de las herramientas más utilizadas para la gestión y manipulación de información estructurada y no estructurada.

Sin embargo, uno de los problemas más recurrentes en este contexto es la ejecución de consultas con errores de sintaxis directamente sobre los sistemas gestores de bases de datos (DBMS). Esta práctica, aunque habitual, conlleva consecuencias negativas que van desde el consumo innecesario de recursos del servidor, pasando por la exposición de datos sensibles ante consultas mal formuladas, hasta la pérdida de tiempo derivada de mensajes de error poco descriptivos que dificultan la comprensión y corrección de los fallos.

A nivel de herramientas disponibles, existen plataformas como SQLFiddle, DB Fiddle y MongoDB Playground, orientadas a la ejecución de consultas en entornos simulados. No obstante, estas herramientas no fueron diseñadas con fines de validación sintáctica pura ni con un enfoque pedagógico: requieren conexión activa a motores de bases de datos reales o simulados, no proporcionan retroalimentación detallada sobre la posición exacta de los errores, y no permiten trabajar con múltiples dialectos SQL y MongoDB en una misma interfaz sin cambiar de plataforma.

En el contexto académico de la Escuela Profesional de Ingeniería de Sistemas de la Universidad Privada de Tacna, los estudiantes del curso de Base de Datos II enfrentan de forma sistemática estas limitaciones al desarrollar sus prácticas de laboratorio y proyectos de desarrollo. La carencia de una herramienta web especializada, gratuita, accesible y orientada al aprendizaje de la sintaxis de consultas motivó el planteamiento del presente proyecto.

La propuesta surge como respuesta directa a esta brecha: desarrollar un sistema web de validación sintáctica que opere de forma independiente de cualquier motor de base de datos, que soporte los dialectos SQL más

utilizados en la industria y las operaciones esenciales de MongoDB, y que provea retroalimentación clara, precisa y pedagógicamente orientada al usuario.

2. Planteamiento del Problema

2.1 Problema

En la actualidad, el proceso más habitual mediante el cual desarrolladores, administradores de bases de datos y estudiantes verifican la corrección de sus consultas SQL o NoSQL consiste en escribirlas directamente en el cliente del sistema gestor de bases de datos de su elección y ejecutarlas contra una base de datos real o de prueba. Esta práctica presenta múltiples inconvenientes:

Dependencia de motores de base de datos activos: Para poder validar una consulta, el usuario necesita tener acceso a una instancia activa del DBMS, lo cual no siempre es posible en contextos académicos donde los estudiantes trabajan desde sus computadoras personales sin servidores configurados localmente.

Mensajes de error poco descriptivos: Los mensajes generados por los distintos DBMS ante errores de sintaxis varían considerablemente en calidad y nivel de detalle. En muchos casos, mensajes como `ERROR 1064 (42000): You have an error in your SQL syntax` no proveen suficiente orientación para corregir el error de forma eficiente.

Riesgos en entornos reales: En producción, ejecutar consultas mal formadas puede generar consecuencias graves. Una sentencia `DELETE` o `UPDATE` sin cláusula `WHERE` por descuido puede modificar o eliminar grandes volúmenes de datos de forma irrecuperable.

Ausencia de herramientas educativas especializadas: No existen actualmente herramientas web de uso gratuito, intuitivas y enfocadas específicamente en la validación sintáctica con fines pedagógicos que soporten múltiples dialectos SQL y MongoDB en una sola interfaz.

Falta de soporte multi-dialecto en una sola herramienta: Los DBMS validan exclusivamente su propio dialecto. No existe una herramienta única que permita validar consultas para múltiples motores sin necesidad de tenerlos todos instalados.

2.2 Justificación

El desarrollo del Validador de Sintaxis SQL u otra se justifica desde múltiples dimensiones:

Justificación técnica: El proyecto aprovecha tecnologías modernas del ecosistema JavaScript —Node.js, Express.js y Monaco Editor— que permiten construir una solución robusta, eficiente y de fácil despliegue. La integración de la librería `node-sql-parser` posibilita un análisis sintáctico de calidad profesional sin necesidad de desarrollar un parser desde cero. La arquitectura bajo el patrón MVC garantiza mantenibilidad y extensibilidad.

Justificación académica: La herramienta actúa como un apoyo pedagógico directo en el proceso de aprendizaje de SQL y NoSQL, reduciendo la barrera de entrada para estudiantes que no disponen de un entorno de base de datos configurado. Al proporcionar mensajes de error claros con indicación de línea y columna, el sistema acelera la comprensión y corrección de errores en el proceso de aprendizaje.

Justificación económica: El proyecto hace uso exclusivo de tecnologías de código abierto, lo que elimina los costos de licenciamiento. El despliegue puede realizarse en plataformas de hosting gratuitas compatibles con Node.js, haciendo que la herramienta sea accesible sin inversión adicional en infraestructura.

Justificación social: La accesibilidad web del sistema lo pone a disposición de cualquier usuario con un navegador moderno, sin importar el sistema operativo o los recursos computacionales disponibles. Esto contribuye a reducir la brecha digital en el acceso a herramientas de aprendizaje de bases de datos.

2.3 Alcance

El sistema contempla las siguientes capacidades dentro de su versión inicial:

Bases de Datos Relacionales (SQL): Validación de sentencias SELECT con cláusulas WHERE, GROUP BY, HAVING, ORDER BY y JOIN; sentencias INSERT, UPDATE y DELETE; con soporte para los dialectos MySQL, PostgreSQL, SQLite y ANSI SQL estándar. Detección de línea y columna exacta del error, con sugerencias de corrección para errores frecuentes.

Bases de Datos No Relacionales (NoSQL — MongoDB): Validación de comandos CRUD (find, insertOne, insertMany, updateOne, updateMany, deleteOne, deleteMany), transacciones, operaciones de sharding, pipelines de agregación con etapas estándar, y operadores especializados como `$mod`, `$text`, `$regex`, `$where`, `$expr`,

`$jsonSchema`, `$geoWithin` y `$near`. Validación de estructura y formato de documentos JSON.

Interfaz web: Editor Monaco con resaltado de sintaxis para SQL y JSON; selector de dialecto SQL; historial de consultas de la sesión; ejemplos precargados por tipo y dialecto; soporte para carga de archivos `.sql` y `.json`; panel de resultados con indicación de errores por línea y columna.

El sistema **no** ejecutará consultas ni se conectará a motores de bases de datos reales. Su función es exclusivamente de validación sintáctica y estructural.

3. Objetivos

3.1 Objetivo General

Desarrollar una aplicación web robusta, funcional y de fácil acceso que permita a estudiantes, desarrolladores y profesionales de tecnología validar la sintaxis de consultas SQL y NoSQL de manera eficiente, precisa y sin necesidad de conexión a un sistema gestor de bases de datos real, contribuyendo a la mejora de la calidad del código de consultas y a la reducción de errores en entornos de desarrollo y producción.

3.2 Objetivos Específicos

- **Implementar la arquitectura MVC:** Organizar el código bajo el patrón Modelo-Vista-Controlador, asegurando separación clara entre lógica de negocio, presentación y flujo de control.
- **Desarrollar un backend escalable con Node.js y Express:** Construir una API REST bien estructurada que exponga servicios de validación con manejo adecuado de errores HTTP, validación de entrada y respuestas estandarizadas en formato JSON.
- **Integrar el editor de código Monaco Editor:** Proporcionar una experiencia de edición de código de calidad profesional, con resaltado de sintaxis, numeración de líneas y capacidad para marcar visualmente los errores directamente en el código.

- **Validar múltiples dialectos SQL:** Implementar la capacidad de validar consultas escritas para MySQL, PostgreSQL, SQLite y ANSI SQL, permitiendo al usuario seleccionar el dialecto de su preferencia.
- **Implementar validación de estructuras NoSQL para MongoDB:** Desarrollar un módulo que valide la corrección sintáctica y estructural de consultas y documentos MongoDB, incluyendo soporte para operadores de consulta avanzados y pipelines de agregación.
- **Proveer retroalimentación detallada al usuario:** Diseñar el sistema para que los mensajes de error sean comprensibles para usuarios con distintos niveles de experiencia, indicando qué está mal, dónde está el error y, cuando sea posible, cómo corregirlo.
- **Facilitar el aprendizaje de SQL y NoSQL en contextos académicos:** Posicionar el sistema como una herramienta pedagógica que apoye a los estudiantes del curso de Base de Datos II en el proceso de aprendizaje iterativo de lenguajes de consulta.

4. Marco Teórico

4.1 Lenguajes de Consulta de Bases de Datos

SQL (Structured Query Language)

SQL es el lenguaje estándar para la gestión de bases de datos relacionales, definido por el estándar ISO/IEC 9075. Permite la definición, manipulación y control de datos almacenados en tablas con relaciones. Las sentencias SQL se clasifican en subconjuntos: DML (Data Manipulation Language) para SELECT, INSERT, UPDATE y DELETE; DDL (Data Definition Language) para CREATE, ALTER y DROP; y DCL (Data Control Language) para GRANT y REVOKE.

Cada motor de base de datos implementa el estándar SQL con variaciones y extensiones propias que dan origen a dialectos específicos: MySQL, PostgreSQL, SQLite, Microsoft SQL Server y Oracle SQL, entre otros.

NoSQL — MongoDB

MongoDB es un sistema de gestión de bases de datos NoSQL orientado a documentos. Almacena datos en formato BSON (Binary JSON), una representación binaria de documentos JSON. Las consultas en MongoDB utilizan una sintaxis basada en documentos JSON, con operadores especiales prefijados por el símbolo `$` (como `$eq`, `$gt`, `$in`, `$and`, `$lookup`). Las operaciones de agregación se realizan mediante pipelines que

encadenan etapas de transformación de datos.

4.2 Análisis Léxico y Sintáctico

El proceso de validación sintáctica se basa en los principios del análisis léxico y sintáctico propios de la teoría de compiladores:

Análisis Léxico (Tokenización): Proceso que descompone una cadena de texto en unidades mínimas con significado propio denominadas tokens. Un token puede ser una palabra reservada (SELECT, FROM, WHERE), un identificador, un operador o un literal. Si se encuentra un carácter que no corresponde a ningún token válido, se genera un error léxico.

Análisis Sintáctico (Parsing): Proceso que verifica si la secuencia de tokens obtenida cumple con las reglas gramaticales del lenguaje. Un parser de SQL verifica, por ejemplo, que una sentencia SELECT esté seguida de una lista de columnas, la cláusula FROM con el nombre de una tabla, y opcionalmente cláusulas como WHERE o ORDER BY. Si la secuencia no satisface las reglas, se genera un error de sintaxis con indicación de la posición del problema.

Árbol de Sintaxis Abstracta (AST): Representación estructurada en árbol de la sintaxis de una consulta, generada por el parser ante una consulta válida. El AST permite extraer información estructural de la consulta más allá de su mera validez sintáctica.

4.3 Patrón de Diseño MVC

El patrón Modelo-Vista-Controlador (MVC) es un patrón de arquitectura de software que separa las responsabilidades de una aplicación en tres componentes:

- **Modelo:** Contiene la lógica de negocio y el procesamiento de datos. En este proyecto, corresponde al módulo `validation.service.js`, que implementa los algoritmos de validación SQL y NoSQL.
- **Vista:** Constituye la interfaz de usuario. En este proyecto, corresponde al frontend (`index.html`, `app.js`,

styles.css) que el usuario ve e interactúa en el navegador.

- **Controlador:** Actúa como intermediario entre la Vista y el Modelo. En este proyecto, corresponde a los archivos `validation.controller.js` y `validate.routes.js` de Express, que reciben las solicitudes HTTP, invocan el servicio de validación y construyen la respuesta.

4.4 Tecnologías Utilizadas

Node.js es una plataforma de ejecución de JavaScript del lado del servidor, basada en el motor V8 de Google Chrome. Su modelo de entrada/salida no bloqueante lo hace especialmente eficiente para aplicaciones de servidor con alta concurrencia.

Express.js es un framework minimalista para Node.js orientado a la construcción de aplicaciones web y APIs REST, que simplifica la definición de rutas, middlewares y el manejo de solicitudes HTTP.

Monaco Editor es el motor de edición de código que potencia Visual Studio Code. Proporciona resaltado de sintaxis, numeración de líneas, autocompletado y la capacidad de marcar errores visualmente en el código mediante su API de decoraciones.

node-sql-parser es una librería npm de código abierto que implementa un parser completo para SQL con soporte para múltiples dialectos. Convierte consultas SQL en un AST y detecta errores sintácticos con información de posición.

API REST (Representational State Transfer) es un estilo de arquitectura de software para servicios web que utiliza el protocolo HTTP y opera sobre recursos identificados por URLs, con operaciones estándar (GET, POST, PUT, DELETE) y respuestas en formato JSON.

5. Desarrollo de la Solución

5.1 Análisis de Factibilidad

5.1.1 Factibilidad Técnica

El proyecto es completamente viable desde el punto de vista técnico. Todas las tecnologías seleccionadas son de código abierto, cuentan con comunidades activas, documentación oficial completa y están ampliamente adoptadas en la industria. Los integrantes del equipo cuentan con conocimientos previos en programación web (HTML, CSS, JavaScript) y desarrollo con Node.js. El proyecto no requiere infraestructura de hardware costosa y puede ser desarrollado en computadoras personales de capacidades estándar.

5.1.2 Factibilidad Económica

Al tratarse de un proyecto académico desarrollado con herramientas de código abierto, los costos totales son considerablemente bajos. El costo total estimado del proyecto es de **S/. 1,693.00**, distribuidos en costos generales (S/. 183.00), costos operativos (S/. 230.00) y costos de personal valorados en 160 horas-hombre (S/. 1,280.00). No se incurre en costos de licenciamiento de software ni de infraestructura de servidores.

5.1.3 Factibilidad Operativa

Al ser una aplicación web, el sistema no requiere ningún proceso de instalación por parte del usuario. Basta con tener un navegador web moderno y acceso a internet. La interfaz está diseñada siguiendo principios de usabilidad web estándar, con un flujo de uso simple: escribir o pegar la consulta, seleccionar el tipo de lenguaje, y hacer clic en “Validar”. La validación de una consulta típica se realiza en menos de 200 milisegundos.

5.1.4 Factibilidad Social

El proyecto tiene un impacto social positivo en el contexto educativo. Provee un recurso de aprendizaje gratuito y accesible para estudiantes de ingeniería de sistemas, reduce la frustración derivada de mensajes de error crípticos de los DBMS, y promueve una cultura de desarrollo más cuidadosa y orientada a la calidad. Al ser una aplicación web, es accesible para estudiantes que no cuentan con los recursos para instalar y configurar un DBMS en sus computadoras personales.

5.1.5 Factibilidad Legal

Todas las librerías y herramientas utilizadas (Node.js, Express.js, Monaco Editor, `node-sql-parser`) están licenciadas bajo la licencia MIT, que permite su uso, modificación y distribución sin restricciones. El código fuente del sistema es

desarrollado íntegramente por el equipo de proyecto. El sistema no solicita, almacena ni procesa información personal de los usuarios, cumpliendo con los principios de la Ley N.º 29733 de Protección de Datos Personales del Perú.

5.1.6 Factibilidad Ambiental

El proyecto tiene un impacto ambiental mínimo. Al ser una aplicación web ligera sobre Node.js, no requiere infraestructura de servidores de alto consumo energético. No genera residuos electrónicos ya que no requiere adquisición de hardware adicional. La reducción de ejecuciones innecesarias sobre servidores de bases de datos contribuye marginalmente a la reducción del consumo de recursos computacionales.

5.2 Tecnología de Desarrollo

Herramienta / Tecnología	Tipo	Versión	Uso en el Proyecto
Node.js	Plataforma de ejecución backend	20.x LTS	Entorno de ejecución del servidor
Express.js	Framework web	4.x	Construcción de la API REST
Monaco Editor	Librería frontend	Última estable	Editor de código con resaltado y marcado de errores
node-sql-parser	Librería npm	v4	Parser SQL multi-dialecto, generación de AST
HTML5 / CSS3 / JavaScript	Tecnologías web	Estándares actuales	Interfaz de usuario (frontend)
npm	Gestor de paquetes	Incluido con Node.js	Gestión de dependencias backend
Git / GitHub	Control de versiones	2.x	Seguimiento de cambios y colaboración
Visual Studio Code	IDE	Última estable	Entorno de desarrollo del equipo
Postman	Herramienta	Última estable	Prueba manual de endpoints de la

	de pruebas		API REST
--	------------	--	----------

5.2.1 Arquitectura del Sistema

El sistema está diseñado bajo una **arquitectura cliente-servidor**, implementada con el patrón **MVC (Modelo-Vista-Controlador)**:

Capa Frontend (Vista): Desarrollada en HTML5, CSS3 y JavaScript vanilla, con el Monaco Editor como componente central. Se comunica con el backend mediante solicitudes HTTP asíncronas al endpoint `POST /api/validate`.

Capa Backend (Controlador + Modelo): Implementada en Node.js con Express.js. Expone una API REST con tres endpoints principales:

- `POST /api/validate` — Recibe `{type, query, dialect?}` y devuelve el resultado de la validación.
- `GET /api/examples` — Devuelve ejemplos de consultas válidas por tipo y dialecto.
- `GET /api/health` — Verifica el estado operativo del servidor.

Estructura de archivos del proyecto:

```

backend/
├── services/
│   └── validation.service.js      # Lógica de validación SQL/NoSQL (Modelo)
├── controllers/
│   └── validation.controller.js   # Manejadores de endpoints (Controlador)
├── middleware/
│   └── errorHandler.middleware.js # Manejo centralizado de errores
│   └── logger.middleware.js      # Registro de solicitudes HTTP
├── routes/
│   └── validate.routes.js        # Definición de rutas de la API
└── server.js                    # Punto de entrada del servidor

frontend/
├── index.html    # Vista principal (Vista)
├── app.js        # Lógica del cliente
└── styles.css    # Estilos de la interfaz

```

5.3 Metodología de Implementación

El proyecto adoptó un enfoque **incremental e iterativo**, organizando el desarrollo en cuatro semanas con entregables verificables al finalizar cada una.

Documento de Visión (FD02)

El Documento de Visión define el posicionamiento del producto, la descripción de los interesados y usuarios, la vista general del producto con sus capacidades, restricciones, rangos de calidad y precedencia de prioridades. Los aspectos clave definidos en este documento que guiaron el desarrollo fueron:

- Identificación de tres perfiles de usuario: principiante, intermedio y avanzado, con necesidades diferenciadas de retroalimentación.
- Definición del sistema como una SPA (Single Page Application) accesible desde el navegador sin instalación.
- Establecimiento de la arquitectura MVC con separación clara entre (modelo), `index.html / app.js` (vista) y `validation.controller.js / validate.routes.js` (controlador).
- Priorización de funcionalidades en cuatro niveles: Crítica (semana 1), Alta (semana 2), Media (semana 3) y Finalización (semana 4).

Documento SRS — Especificación de Requerimientos de Software (FD03)

El documento SRS formalizó los requerimientos funcionales y no funcionales del sistema, los perfiles de usuario, los casos de uso principales y las reglas de negocio. Los requerimientos más relevantes incorporados en el desarrollo fueron:

Requerimientos funcionales principales:

ID	Requerimiento	Prioridad
RF-01	Validar sintaxis SQL con soporte para MySQL, PostgreSQL, SQLite y ANSI	Alta
RF-02	Retornar línea y columna exacta de cada error sintáctico SQL	Alta
RF-03	Proveer sugerencias de corrección para errores SQL comunes	Media
RF-04	Validar estructura JSON válida en consultas MongoDB	Alta
RF-05	Validar comandos MongoDB soportados (CRUD,	Alta

	aggregate, etc.)	
RF-06	Validar operadores MongoDB (\$in, \$and, \$or, \$mod, \$text, etc.)	Alta
RF-07	Ofrecer editor Monaco con resaltado de sintaxis para SQL y JSON	Alta
RF-08	Guardar cada validación en el historial local del usuario	Media
RF-09	Permitir cargar archivos .sql y .json para validación	Media
RF-10	Exponer endpoint POST /api/validate para validación programática	Alta

Requerimientos no funcionales principales:

ID	Categoría	Requerimiento
RNF-01	Rendimiento	Validación en menos de 2 segundos para consultas de hasta 10,000 caracteres
RNF-02	Disponibilidad	Backend con disponibilidad del 99% en producción
RNF-03	Usabilidad	Interfaz intuitiva sin requerir manual de usuario
RNF-04	Compatibilidad	Funcional en Chrome 90+, Firefox 88+, Safari 14+ y Edge 90+
RNF-05	Mantenibilidad	Patrón MVC con cobertura de pruebas unitarias $\geq 80\%$
RNF-06	Seguridad	Sanitización de entradas para prevenir inyección de código
RNF-07	Portabilidad	Backend ejecutable en Node.js v18+ en Linux, macOS y Windows
RNF-08	Escalabilidad	Soporte de al menos 100 peticiones/segundo sin degradación

Casos de uso principales identificados:

- **UC-01 — Validar Consulta SQL:** El usuario selecciona el tipo SQL, elige el dialecto, escribe la consulta en el editor Monaco y presiona Validar. El sistema envía al endp `POST /api/validate`, el parser procesa la consulta y retorna `{valid, errors:[{line, column, message}], suggestions}`. El resultado es mostrado en pantalla y guardado en el historial local.
- **UC-02 — Validar Consulta MongoDB:** El usuario selecciona el tipo NoSQL, ingresa el documento JSON MongoDB y presiona Validar. El sistema valida la estructura JSON, el comando MongoDB y los operadores utilizados, retornando errores específicos por tipo.
- **UC-03 — Cargar Archivo:** El usuario presiona “Subir Archivo” y selecciona un archivo `.sql` o `.json`. El editor carga el contenido automáticamente y ajusta el modo de validación correspondiente.
- **UC-04 — Consultar Historial:** Cada validación se guarda en `localStorage`. El botón “Historial” muestra la lista de consultas anteriores de la sesión.

Reglas de negocio aplicadas:

ID	Regla
RN-01	Toda validación SQL debe especificar un dialecto (MySQL por defecto si no se especifica)
RN-02	Toda consulta MongoDB debe ser JSON válido antes de validar comandos y operadores
RN-03	Los operadores <code>\$in</code> , <code>\$nin</code> , <code>\$all</code> , <code>\$and</code> , <code>\$or</code> , <code>\$nor</code> deben tener como valor un array
RN-04	El operador <code>\$mod</code> debe tener exactamente 2 elementos: <code>[divisor, residuo]</code>
RN-05	El operador <code>\$text</code> debe ser un objeto que contenga la propiedad <code>\$search</code> de tipo string
RN-06	El historial se almacena en <code>localStorage</code> y no se sincroniza entre dispositivos
RN-07	Solo se aceptan archivos con extensión <code>.sql</code> o <code>.json</code> para carga desde el sistema de archivos

Documento de Arquitectura de Software (FD04)

El documento SAD describió la arquitectura del sistema utilizando el modelo 4+1, con las siguientes vistas:

Vista de Casos de Uso: Identificó dos actores principales (Usuario y Sistema) y seis casos de uso: validar consulta SQL, validar consulta NoSQL, mostrar errores, seleccionar lenguaje, cargar archivo y ver ejemplos.

Vista Lógica (MVC): Definió la estructura interna del sistema bajo el patrón MVC con los subsistemas Frontend, Backend, Controladores, Servicios y Middleware, y sus relaciones de comunicación mediante solicitudes HTTP y delegaciones internas.

Vista de Implementación: Organizó el código en cuatro capas funcionales: Presentación (Frontend), Aplicación (Controllers), Dominio (Services) e Infraestructura (Parser), con componentes bien delimitados.

Vista de Procesos: Describió el flujo completo del sistema: acceso del usuario → ingreso de consulta → validación → decisión (error o correcto) → presentación del resultado.

Vista de Despliegue: Especificó la distribución física del sistema con el cliente web en el navegador del usuario y el servidor Node.js desplegable localmente o en plataformas de hosting gratuitas como Render, Railway o Vercel.

Atributos de Calidad definidos:

- **Funcionalidad:** Validación correcta de SQL multi-dialecto y MongoDB con retroalimentación precisa.
- **Usabilidad:** Interfaz web basada en Monaco Editor, intuitiva y sin requerimiento de instalación.
- **Confiabilidad:** No se almacenan datos sensibles; las consultas no modifican bases de datos reales; el sistema no colapsa ante entradas maliciosas.
- **Rendimiento:** Respuesta en milisegundos para consultas estándar; soporte de múltiples solicitudes simultáneas.
- **Mantenibilidad:** Arquitectura MVC que permite agregar nuevos módulos de validación sin modificar el núcleo del sistema.
- **Escalabilidad:** Backend Node.js con manejo asíncronico de solicitudes, preparado para escalar horizontalmente.

6. Cronograma

El proyecto fue desarrollado en un período de **cuatro semanas** durante el ciclo académico 2026-I, siguiendo la distribución planificada:

Semana	Período	Actividades	Entregable
Semana 1	28/03/2026 — 04/04/2026	Levantamiento de requerimientos. Análisis de herramientas. Diseño de arquitectura MVC. Definición del stack tecnológico. Elaboración de diagramas. Configuración del proyecto y estructura de directorios. Servidor Express base. Módulo de validación SQL (MySQL) funcional.	Documento de Factibilidad (FD01). Proyecto configurado con validación SQL base operativa.
Semana 2	05/04/2026 — 18/04/2026	Extensión del módulo SQL a todos los dialectos (PostgreSQL, SQLite, ANSI). Desarrollo del módulo de validación NoSQL MongoDB (CRUD, operadores, aggregation pipeline). Implementación de endpoints GET /api/health y GET /api/examples. Aseguramiento de detección de línea y columna en errores SQL.	Documento de Visión (FD02). API REST completamente funcional con validación SQL multi-dialecto y NoSQL.

Semana 3	19/04/2026 — 02/05/2026	Diseño y maquetación del frontend. Integración del Monaco Editor desde CDN. Conexión frontend-backend con mecanismo debounce. Implementación del marcado visual de errores en Monaco. Desarrollo del panel de resultados.	Documento SRS (FD03). Interfaz web completamente operativa y conectada al backend.
Semana 4	03/05/2026 — 30/05/2026	Historial de consultas de la sesión. Ejemplos precargados en la interfaz. Carga de archivos .sql y .json. Pruebas de integración end-to-end. Corrección de bugs. Documentación técnica (README, JSDoc). Elaboración del informe final.	Documento SAD (FD04). Sistema completo, probado y documentado. Informe Final (FD05).

7. Presupuesto

7.1 Costos Generales

Ítem	Cantidad	Costo Unitario (S/.)	Total (S/.)
Materiales de oficina (cuaderno S, lapiceros)	2 paquetes	10.00	20.00

Útiles varios (post- its, marcadore s)	4 unidades	2.00	8.00
Impresión y empastad o del informe final	1 juego	80.00	80.00
Servicio de internet (cuota mensual por integrante)	1 mes	50.00	50.00
Almacena miento en la nube (Google Drive)	1 mes	25.00	25.00
TOTA L COST OS GENERA LES			183.00

7.2 Costos Operativos

Concepto	Costo (S/.)
Consumo de energía eléctrica	40.00
Servicio de internet adicional	50.00
Transporte (movilidad para reuniones)	60.00
Alimentación durante jornadas de trabajo	80.00
TOTAL COSTOS OPERATIVOS	230.00

7.3 Costos de Personal

Rol	Horas	Pago/Hora (S/.)	Total (S/.)
-----	-------	-----------------	-------------

Soto Oquendo Cristian Gabriel — Especialista Backend	80 h	8.00	640.00
Arocuti pa Arocuti pa Gian Franco — Especialista Frontend	80 h	8.00	640.00
TOTAL COSTOS DE PERSONAL	160 h		1,280.00

7.4 Resumen del Presupuesto

Tipo de Costo	Monto (S/.)	Porcentaje
Costos Generales	183.00	10.81%
Costos Operativos	230.00	13.58%
Costos de Personal	1,280.00	75.61%
COSTO TOTAL DEL PROYECTO	1,693.00	100%

8. Conclusiones

1. **El proyecto fue desarrollado exitosamente dentro del plazo académico establecido.** El sistema Validador de Sintaxis SQL u otra fue construido, integrado y documentado en cuatro semanas, cumpliendo con todas las funcionalidades de prioridad crítica y alta definidas en el documento de Visión y el SRS, incluyendo la validación SQL multi-dialecto, la validación NoSQL para MongoDB, y la interfaz web con Monaco Editor.

2. **La arquitectura MVC demostró ser la elección correcta para este tipo de sistema.** La separación clara entre el módulo de validación (`validation.controller.js`), la interfaz de usuario (`index.html` , `app.js`) y los controladores de la API (`validation.controller.js` , `validate.routes.js`) facilitó el desarrollo paralelo del frontend y el backend, redujo los conflictos de integración y garantizó que la adición de nuevas capacidades de validación no impacte el resto de la arquitectura.
3. **La integración de `node-sql-parser v4` proveyó una base de análisis sintáctico robusta y multi-dialecto.** La librería permitió implementar la validación SQL para los cuatro dialectos soportados (MySQL, PostgreSQL, SQLite y ANSI) con alta precisión en la detección de errores y generación de información de posición (línea y columna),

superando las expectativas iniciales en cuanto a calidad de retroalimentación.

4. El Monaco Editor elevó significativamente la calidad de la experiencia de usuario. La integración del mismo motor de edición de Visual Studio Code en el frontend proporcionó una experiencia profesional con resaltado de sintaxis, numeración de líneas y marcado visual de errores directamente en el código, comparable a la de un IDE de desarrollo.

5. El sistema cumple eficazmente su función pedagógica. Al proporcionar mensajes de error claros con indicación de línea, columna y sugerencias de corrección, sin necesidad de conexión a un motor de base de datos real, el sistema elimina las principales barreras en el proceso de aprendizaje autónomo de SQL y NoSQL para estudiantes del curso de Base de Datos II.

6. El costo total del proyecto fue mínimo y justificado. Con un costo estimado de S/. 1,693.00 —del cual el 75.61% corresponde a la valorización del tiempo de los desarrolladores— el proyecto representa una inversión mínima con un alto retorno en términos de valor educativo y de reducción de riesgos en el desarrollo de consultas.

7. El sistema sienta bases sólidas para futuras extensiones. La arquitectura modular y el diseño del servicio de validación como punto único de extensión permiten agregar soporte para nuevos dialectos SQL, nuevos motores NoSQL (Cassandra, Redis), validación semántica avanzada o autenticación de usuarios en versiones futuras, sin necesidad de rediseñar la arquitectura existente.

9. Recomendaciones

1. Ampliar el soporte de dialectos SQL en versiones futuras. La versión actual soporta MySQL, PostgreSQL, SQLite y ANSI. Se recomienda incorporar soporte para Microsoft SQL Server y Oracle SQL, dialectos ampliamente utilizados en entornos empresariales, lo que ampliaría significativamente el alcance de la herramienta.

2. Implementar persistencia del historial en servidor. El historial de consultas actual se almacena exclusivamente en `localStorage` del navegador, lo que implica pérdida de datos al cambiar de dispositivo o limpiar el almacenamiento. Se recomienda implementar en versiones futuras un sistema de autenticación con persistencia del historial en base de datos, habilitando el acceso desde múltiples dispositivos.

3. Desarrollar pruebas unitarias con mayor cobertura para el módulo de validación NoSQL. Dada la mayor flexibilidad estructural de

MongoDB y la complejidad de sus operadores avanzados, se recomienda ampliar el conjunto de pruebas unitarias del módulo `validation.service.js` para los casos de validación NoSQL, con énfasis en los operadores de aggregation pipeline y los operadores geoespaciales.

4. Incorporar el sistema como herramienta de apoyo formal en el currículo del curso. Se recomienda a la cátedra de Base de Datos II de la EPIS-UPT evaluar la integración del sistema como recurso didáctico oficial en las sesiones de laboratorio, especialmente en las unidades dedicadas a la escritura de consultas DML y a las operaciones de agregación en MongoDB.

5. Explorar la integración con entornos de CI/CD. La API REST bien documentada del sistema permite su integración como etapa de validación en pipelines de integración continua (CI/CD), donde las consultas SQL y NoSQL de un proyecto podrían ser validadas automáticamente antes de ser desplegadas. Se recomienda documentar casos de uso específicos para este escenario.

6. Agregar soporte para validación semántica básica. La versión actual valida exclusivamente la corrección sintáctica de las consultas. Una mejora de alto valor sería la implementación de validación semántica básica: verificar que los tipos de datos utilizados en las cláusulas WHERE sean compatibles con los operadores aplicados, o que las funciones de agregación sean usadas correctamente con cláusulas GROUP BY.

7. Diseñar un modo de práctica guiada. Se recomienda incorporar en versiones futuras un módulo de ejercicios interactivos donde el usuario reciba una descripción de la consulta que debe construir y el sistema evalúe si la consulta ingresada corresponde a la solución esperada, combinando la validación sintáctica existente con una comparación estructural del AST generado.

10. Bibliografía

- Informe de Factibilidad del Proyecto — Documento FD01, versión 1.0, elaborado por Soto / Arocutipa, aprobado el 28/03/2026.
- Documento de Visión — FD02, versión 1.0, elaborado por Soto / Arocutipa, aprobado el 28/03/2026.
- Especificación de Requerimientos de Software — FD03, versión 1.0, elaborado por Arocutipa / Soto, 27/04/2026.
- Documento de Arquitectura de Software — FD04, versión 1.0, elaborado por Soto / Arocutipa, aprobado el

28/04/2026.

- Documentación oficial de SQL estándar ANSI — ISO/IEC 9075. Organismos de estándares internacionales.
- Documentación oficial de MongoDB. Disponible en: <https://www.mongodb.com/docs/>
- Documentación oficial de Node.js v20 LTS. Disponible en: <https://nodejs.org/docs/>
- Documentación oficial de Express.js v4. Disponible en: <https://expressjs.com/>
- Repositorio oficial de node-sql-parser v4. Disponible en: <https://github.com/taozhi8833998/node-sql-parser>
- Documentación del Monaco Editor. Disponible en: <https://microsoft.github.io/monaco-editor/>
- Aho, A. V., Lam, M. S., Sethi, R., & Ullman, J. D. (2007). *Compilers: Principles, Techniques, and Tools* (2nd ed.). Pearson Education.
- Material académico del curso de Base de Datos II — Universidad Privada de Tacna, Escuela Profesional de Ingeniería de Sistemas, ciclo 2026-I. Mag. Patrick Cuadros Quiroga.

11. Anexos

Anexo 01 — Informe de Factibilidad (FD01)

Documento que analiza la viabilidad técnica, económica, operativa, legal, social y ambiental del proyecto. Contiene la descripción del proyecto, la identificación de riesgos con sus estrategias de mitigación, el análisis de la situación actual del problema, el estudio de factibilidad completo y el análisis financiero con indicadores B/C, VAN y TIR. Versión 1.0, aprobada el 28/03/2026.

Anexo 02 — Documento de Visión (FD02)

Documento que define la visión del producto, el posicionamiento, la descripción de los interesados y usuarios, la vista general del producto con sus capacidades, restricciones, rangos de calidad y precedencia de prioridades de implementación. Versión 1.0, aprobada el 28/03/2026.

Anexo 03 — Documento SRS — Especificación de Requerimientos de Software (FD03)

Documento que formaliza los requerimientos funcionales y no funcionales del sistema, los perfiles de usuario, los casos de uso principales con sus escenarios detallados, el modelo conceptual con diagramas de paquetes y casos de uso, el modelo lógico con diagramas de actividades, secuencia y clases, y las reglas de negocio del sistema. Versión 1.0, elaborada el 27/04/2026.

Anexo 04 — Documento de Arquitectura de Software (FD04)

Documento que describe la arquitectura del sistema bajo el modelo 4+1, incluyendo la vista de casos de uso, vista lógica (MVC), vista de implementación, vista de procesos y vista de despliegue. Contiene las tablas de requerimientos funcionales y no funcionales priorizados, restricciones del proyecto y atributos de calidad del software. Versión 1.0, aprobada el 28/04/2026.

Anexo 05 — Manual de Usuario y README del Repositorio

Documentación técnica que incluye las instrucciones de instalación y ejecución local del sistema, la descripción de los endpoints de la API REST con ejemplos de solicitudes y respuestas en formato JSON, los dialectos SQL y operaciones MongoDB soportadas, y ejemplos de consultas válidas e inválidas para cada tipo de lenguaje. Disponible en el repositorio del proyecto en GitHub.

Fin del Informe Final — Versión 1.0

Universidad Privada de Tacna — Facultad de Ingeniería — Escuela

Profesional de Ingeniería de Sistemas Curso: Base de Datos II —

Docente: Mag. Patrick Cuadros Quiroga — Año: 2026